

Semaine 10 · Cours 1

Récapitulatif Général & Lancement du Projet 1

GL-EN3-2026 · Module 2 → Module 3 · Lundi 1er juin 2026

Jean Fritz SAINT-PAUL · FDS-UEH

Plan du cours

1 Rappel Objectif du cours

Pourquoi vous êtes là · Le cycle de vie de l'ingénieur logiciel

2 Récapitulatif Modules 1 & 2

De l'esprit d'ingénieur à la conception complète

3 Prochaines étapes & Projet 1

Calendrier · Module 3 · Lancement officiel

Objectif du cours

Partie 1 — Pourquoi vous êtes là

Transformer les ingénieurs électroniciens EN3 en ingénieurs logiciels modernes, capables de concevoir, développer et déployer des systèmes intelligents qui résolvent les vrais problèmes d'Haïti.

Ce n'est pas un cours de code — c'est un cours de livraison de valeur.

Codeur exécutant	Ingénieur logiciel
Reçoit un cahier des charges	Comprend le problème d'abord
Livre une fonctionnalité	Livre de la valeur mesurable
Code sans comprendre le contexte	Questionne les hypothèses
Son travail s'arrête au commit	Suit l'impact en production
Remplaçable par l'IA en 2026	Augmenté par l'IA en 2026 ✓

Complexité essentielle vs accidentelle

Fred Brooks, No Silver Bullet (1986) — confirmé par l'IA de 2026

Complexité ESSENTIELLE

- Comprendre le problème : explorer les besoins réels, interviewer les utilisateurs
- Modéliser le système : concevoir l'architecture, abstraire la complexité
- Choisir la bonne solution : compromis valeur / coût / risque, vision système
- Coordonner les équipes : communication, décision, planification

→ Ce que l'IA ne peut pas faire à votre place

Complexité ACCIDENTELLE

- Écrire le code : traduction de la solution en instructions machine
- Compiler et déboguer : largement automatisé
- Configurer les environnements : infrastructure, déploiement
- Tâches répétitives : boilerplate, tests unitaires, documentation

→ Ce que les outils (et l'IA) prennent en charge

« L'IA code mieux que vous en 2026. Mais elle ne comprend pas le problème. Votre valeur = maîtriser l'essentiel. »

Le cycle de vie de l'ingénieur logiciel

10 étapes · mapping sur les 4 modules du cours

Étape	Nom	Description	Ce cours
01	Espace des problèmes	Explorer les douleurs · Ne pas sauter à la solution	Module 1
02	Choix du problème	Sélectionner LE problème · Valeur et faisabilité	Module 1
03	Conception	UX · Architecture · Wireframes · Prototypes	Module 2
04	Développement	Itérations courtes · TDD · Revues de code	Modules 3–4
05	Tests & Validation	Tests unitaires · Intégration · UAT	Modules 3–4
06	Déploiement	Mise en production · CI/CD · Monitoring initial	Modules 3–4
07	Mesure	Métriques · Retours utilisateurs · A/B testing	Dès la conception
08	Apprentissage	Rétrospective · Backlog enrichi · Patterns	Dès la conception
09	Maintenance	Bugs · Sécurité · Améliorations continues	Dès la conception
10	Évolution / Refonte	Le produit grandit ou est refondu selon le marché	Dès la conception

Structure révisée du cours

4 modules · 2 projets

Les 4 modules

#	Titre	Contenu clé	Statut
1	L'esprit d'ingénieur	Interviews · CJM · Pain Point · Hypothèse · MVP	✅ Terminé
2	Analyse et Conception	User Stories · UML · ERD · Architecture · Sécurité	✅ Terminé
3	Travailler en équipe — Agilité	Scrum · Kanban · Git · Pull Requests · DoD · DoR	📅 Suivant
4	Développement assisté par IA	Agents IA · Prompting · Tests · Validation · Livraison	📅 À venir

Les 2 projets

Projet	Contenu	Pondération	Échéance
Projet 1	Cahier des charges complet	35 %	Vendredi 19 juin 2026
Projet 2	Développement assisté par IA	60 %	Fin Module 4
Participation	Présence + Cours en ligne	5 – 10 %	Fin Module 4

Les 4 équipes

Capitaines · Membres · Modules FDS assignés

#	Équipe	Capitaine	Membres	Module FDS
1	Brainstorm	NICOLAS	NICOLAS Jean Nickson · SAINT GERMAIN Pierre Michel Junior · JEAN CHARLES Carlbens Jovani Junior	M4 — FDS Pay
2	Sterno X-Core Group	NOEL	JEAN-BAPTISTE Olivier Stéphane · NOEL Richardson Wisler · VIXAMAR Sergelie	M1 — FDS Akademi
3	Neural Squad	CHARLES	CHARLES Jean Guetcheen · JOSPEPH Wiycleph · PIERRE Rose Sona Ediasqua	M3 — FDS Admin
4	Bravetech	VAILLANT	DORFILUS Skin-Paolatchi · GUERRIER Joas · LOUIS Widmaken · VAILLANT Valcin	M2 — FDS Portail

Les modules FDS et leurs pain points

Persona · Problème réel · Équipe assignée

Module	Équipe	Persona	Pain point principal
M1 — FDS Akademi	Sterno X-Core Group	Étudiant FDS	Un étudiant ne peut pas prouver ni attester des compétences qu'il déclare dans son CV ou lors d'un entretien
M2 — FDS Portail	Bravetech	Candidat à l'inscription	Un lycéen qui veut s'inscrire à la FDS ne peut pas trouver une information officielle et fiable sans se déplacer
M3 — FDS Admin	Neural Squad	Étudiant FDS	Un étudiant qui dépose une demande administrative à la FDS n'a aucune visibilité sur ce qui se passe après avoir posé son dossier
M4 — FDS Pay	Brainstorm	Étudiant FDS	Un étudiant n'a aucun moyen de payer ses frais à la FDS sans se déplacer avec du cash ou sans dépendre d'un intermédiaire

Récapitulatif

Module 1

L'esprit d'ingénieur

Ce qu'on a vu — Module 1

L'esprit d'ingénieur en génie logiciel

1

Comprendre le problème

Avant d'écrire une seule ligne de code, vérifier que le problème a de la valeur. L'utilisateur souffre-t-il vraiment ? Est-il prêt à changer ses habitudes ?

2

Interviews & Customer Journey Map (CJM)

Interviews : poser les bonnes questions, écouter les verbatims.
CJM : 6 composantes — étapes · actions · émotions · pain points · opportunités · moments de vérité.

3

De l'observation à l'hypothèse

Format 4 parties : « Nous croyons que [persona] a besoin de [solution] pour [résultat]. Nous saurons que c'est vrai si [critère mesurable]. »

4

Le MVP — Build to Learn

Minimum Viable Prototype : tester l'hypothèse avec le moins de ressources possible. Révèle ce que les utilisateurs font réellement, pas ce qu'ils disent.

Livrable Module 1 : Hypothèse testable + Walking Skeleton → entrée du Module 2

Récapitulatif

Module 2

Analyse et Conception

Ce qu'on a vu — Module 2

L'esprit d'ingénieur en génie logiciel

1

Du MVP aux User Stories

JTBD · Format INVEST · Priorisation MoSCoW

2

UML : 5 diagrammes essentiels

Cas d'utilisation · Classe · Séquence · Activité · Composants

3

Modélisation des données

Langage ubiquitaire · 3 niveaux · SQL vs NoSQL · System of Record

4

Architecture technique

3-tiers vs MVC · Spectre des architectures · 5 styles d'API · Contract-First

5

Sécurité par conception

Security by Design · OWASP Top 10:2025 · AuthN vs AuthO · JWT · RBAC vs ABAC

6

Performance & Accessibilité

3 leviers de performance · Règle 80/20 · WCAG 2.1 P.O.U.R. · Cible AA

7

Revue architecturale — ADR & ATAM

Documenter et évaluer les décisions d'architecture

Du MVP aux User Stories

JTBD · Format INVEST · Priorisation MoSCoW

Job to be Done (JTBD)

« Quand je [situation critique], je veux [résultat fonctionnel] afin de [résultat dans ma vie]. »

User Stories — format INVEST

« En tant que [persona], je veux [action] afin de [bénéfice]. »
Critères Gherkin : « Lorsque [précondition], quand [action], alors [résultat attendu]. »

Priorisation MoSCoW

Must Have	— Indispensable pour valider l'hypothèse
Should Have	— Important mais non bloquant pour le MVP
Could Have	— Utile si le temps le permet
Won't Have	— Explicitement hors périmètre pour cette version

À retenir

→ Le JTBD n'est pas une fonctionnalité — c'est un changement de situation pour l'utilisateur.

→ Une User Story Must Have doit avoir des critères d'acceptation Gherkin.

→ MoSCoW force l'équipe à trancher — rien n'est « prioritaire par défaut ».

→ Le Walking Skeleton est le fil conducteur : la fonctionnalité la plus risquée, bout en bout.

UML : 5 diagrammes essentiels

Unified Modeling Language — un outil, une question

Diagramme	Question posée	Lien avec le projet
Cas d'utilisation	<i>Qui fait quoi dans le système ?</i>	User Stories → Use Cases → acteurs et interactions
Classes	<i>Quelle est la structure des objets métier ?</i>	Entités → ERD → tables SQL
Séquence	<i>Dans quel ordre les composants échangent-ils ?</i>	Vérifier que chaque scénario est réalisable
Activités	<i>Comment un processus complexe se déroule-t-il ?</i>	Algorithmes et flux avec conditions et boucles
Composants	<i>Comment les blocs du système sont-ils organisés ?</i>	Architecture technique et interfaces entre modules

Règle d'or : chaque diagramme répond à une question précise. Ne pas en abuser — 3 à 5 diagrammes bien choisis valent mieux que 15 diagrammes vides de sens.

Modélisation des données

Langage ubiquitaire · 3 niveaux · SQL vs NoSQL · System of Record

3 niveaux d'abstraction

Conceptuel

Entités & relations du domaine
Langage naturel · Fact-Based Modeling

Logique

Structure formalisée indépendante du SGBD
ERD (entités, attributs, cardinalités, PK/FK)

Physique

Implémentation dans un SGBD précis
CREATE TABLE avec contraintes et index

Normalisation 1NF → 2NF → 3NF

1NF : pas de groupes répétés, attributs atomiques

2NF : chaque attribut dépend de toute la clé

3NF : pas de dépendance transitive

SQL vs NoSQL

Critère	SQL (relationnel)	NoSQL
Cohérence	ACID — forte	BASE — éventuelle
Cas d'usage	Données structurées, transactions	Données flexibles, haute volumétrie
Projet 1	✅ Par défaut	Justifier si choisi

System of Record

Pour chaque donnée partagée entre modules : quel module est la source de vérité ?

Les autres modules consomment — ils ne dupliquent pas.

Le module SYS est le System of Record de toutes les identités utilisateur de la plateforme FDS.

Architecture technique

3-tiers vs MVC · Spectre des architectures · 5 styles d'API · Contract-First

Spectre des architectures

Architecture	Complexité	Équipe	Cas d'usage
Monolithe ★	Faible	1–5 pers.	MVP, prototype
Monolithe Modulaire	Faible	3–10 pers.	Domaine établi, maintenabilité
Microservices	Élevée	10+ pers.	Grande org., scalabilité diff.
Serverless	Gérée cloud	Variable	Event-driven, tâches périodiques

★ *Recommandation Projet 1 : Monolithe Commencez simple. Migrez quand la douleur est réelle — pas avant.*

3-tiers vs MVC

3-tiers = déploiement physique (Présentation → Logique → Données). MVC = organisation interne du code. Les deux coexistent.

5 styles d'API

Style	Protocole	Cas d'usage
REST ★	HTTP/HTTPS	CRUD, APIs publiques, mobile backends
GraphQL	HTTP	UI flexible, besoins variés côté client
gRPC	HTTP/2	Microservices internes, haute performance
WebSockets	TCP	Chat, collaboration temps réel, jeux
SSE	HTTP	Notifications push, dashboards live

Contract-First & OpenAPI

Définir le contrat API avant de coder. openapi.yaml = référence partagée frontend/backend.

Sécurité par conception

Security by Design · OWASP Top 10:2025 · AuthN vs AuthO · JWT · RBAC

Least Privilege

Chaque composant n'accède qu'à ce dont il a strictement besoin

Defense in Depth

Plusieurs couches de protection indépendantes

Fail Secure

En cas d'erreur, refuser l'accès — jamais l'autoriser par défaut

Code	Vulnérabilité	Mitigation principale
A01	Broken Access Control	Deny by default · RBAC · vérification côté serveur
A02	Security Misconfiguration	.env pour les secrets · headers HTTP · désactiver le debug
A03	Software Supply Chain Failures ★	npm audit · SBOM · revue CI/CD obligatoire
A04	Cryptographic Failures	bcrypt · HTTPS · AES-256 · secrets hors du repo Git
A05	Injection	Requêtes paramétrées · ORM · validation côté serveur
A06	Insecure Design	Threat Modeling · misuse cases · rate limiting
A07	Authentication Failures	MFA · rate limiting login · librairie d'auth éprouvée
A08	Software / Data Integrity Failures	Signatures numériques · SRI · revue de code
A09	Security Logging & Alerting Failures	Logs structurés · centralisation · alertes
A10	Mishandling of Exceptional Conditions ★	Fail closed · rollback · messages d'erreur génériques

Projet 1 : AuthN = JWT · AuthO = RBAC. Tables : users · roles · user_roles. | AuthN « Qui es-tu ? » → JWT. AuthO « Que peux-tu faire ? » → 200 ou 403.

Performance & Accessibilité

3 leviers de performance · Règle 80/20 · WCAG 2.1 P.O.U.R. · Cible AA

3 leviers de performance

Levier	Principe	Outil
Caching	Ne pas recalculer ce qui ne change pas	Redis · Cache-Aside · Cache-Control HTTP
Indexation	Accélérer les recherches fréquentes	Index B-Tree sur colonnes WHERE / JOIN / ORDER BY
Scalabilité	App stateless pour ajouter des instances	Scale Out horizontal · Load Balancer

*Règle 80/20 : 80% des problèmes viennent de 20% du code.
Mesurer avant d'optimiser — ne jamais optimiser à l'aveugle.*

Accessibilité — WCAG 2.1 P.O.U.R.

	Principe	Exemple
P	Perceptible	Alt text · contraste 4.5:1 min. · sous-titres
O	Opérable	Navigation clavier complète · focus visible
U	Understandable	Labels explicites · messages d'erreur clairs
R	Robust	HTML sémantique · ARIA roles

Cible Projet 1 : WCAG 2.1 niveau AA — standard recommandé dans la plupart des contextes professionnels.

À concevoir dès l'architecture — pas en fin de projet

Performance mal conçue = dette technique coûteuse.

Accessibilité oubliée = refonte de l'interface. Les deux se décident dans la section 11 de votre cahier des charges.

Revue architecturale — ADR & ATAM

Documenter et évaluer les décisions d'architecture

Format ADR — Architectural Decision Record

Conséquences :

Titre : [Décision]

Contexte : Pourquoi cette décision est nécessaire

Décision : Ce qui a été choisi et POURQUOI

+ Impact positif

- Risque à surveiller

ATAM simplifié — 5 étapes

1

Présenter l'architecture

Diagramme de composants, flux principaux

2

Lister les attributs qualité

Performance, sécurité, maintenabilité, accessibilité

3

Identifier les décisions clés

API style, ORM, authentification, BDD

4

Analyser les compromis

Trade-offs pour chaque décision

5

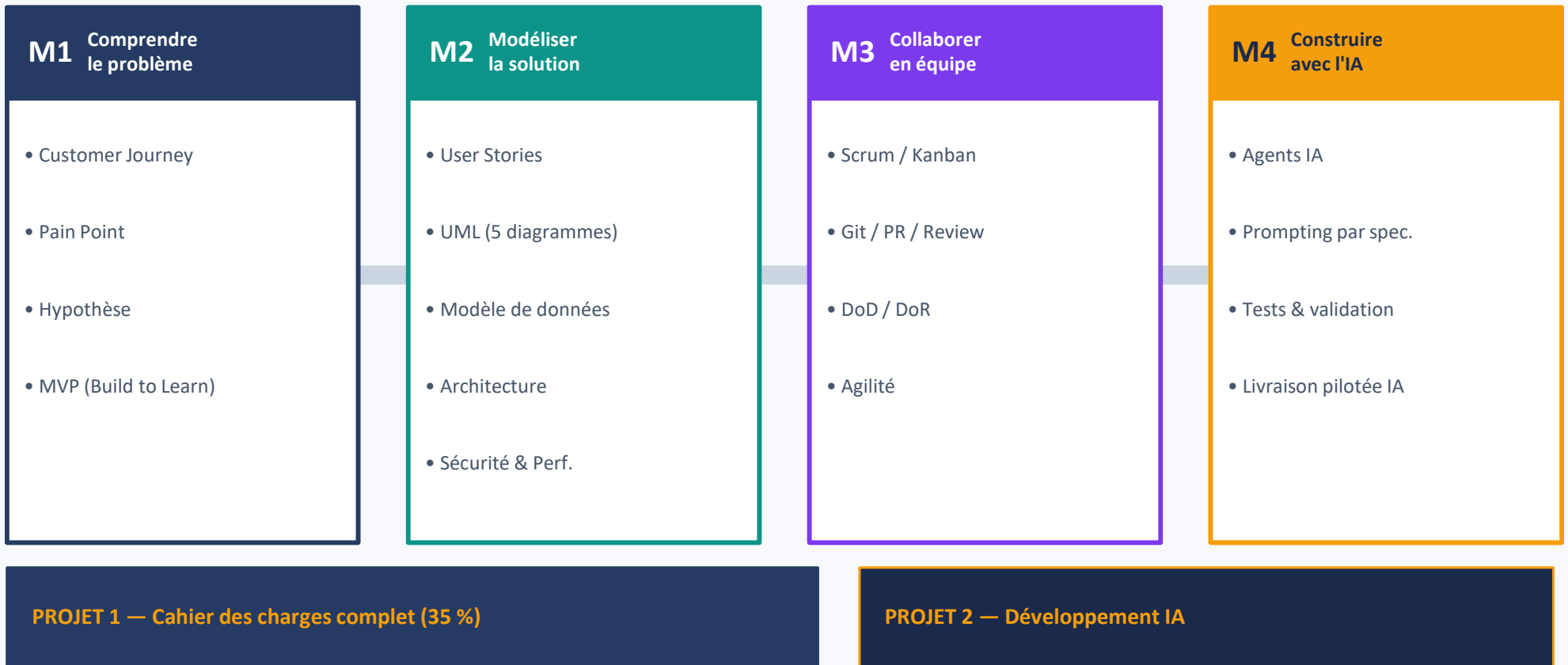
Documenter en ADR

Au minimum 1 ADR par décision majeure

Minimum requis : 1 ADR rédigé pour la décision architecturale principale dans la section 11.

La ligne directrice

Vue complète des 4 modules en séquence



Prochaines étapes

& Projet 1

Calendrier · Module 3 · Lancement officiel

Projet 1 — Cahier des charges complet

Pondération : 35 % · Rendu : fin du Module 2

Qu'est-ce que le Projet 1 ?

Produire le cahier des charges complet de votre module. C'est la formalisation de tout ce que vous avez appris dans les modules 1 et 2 — du problème à l'architecture.

35 %

du cours

A Votre module, votre contexte

Le cahier des charges est spécifique à votre module FDS. Chaque section découle logiquement de la précédente.

B Fondation du Projet 2

Le Projet 2 prend votre cahier des charges comme entrée et génère le code avec un agent IA. Un document précis = un code précis.

C Cohérence évaluée

Ce qui est évalué, c'est la cohérence : problème → persona → hypothèse → MVP → use cases → modèle → architecture.

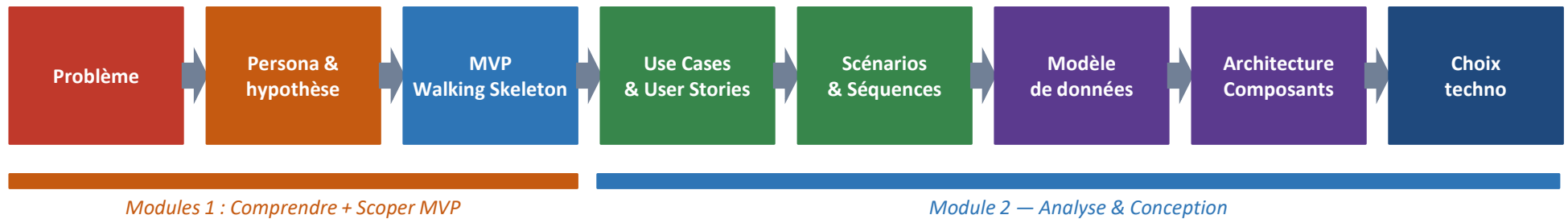
Structure du cahier des charges — 11 sections

Dans l'ordre · Chaque section découle de la précédente

1	Le problème <i>Décrire le problème réel — pas encore la solution</i>	6	Use Cases & User Stories <i>Diagramme UML · User Stories · Descriptions détaillées</i>
2	La solution proposée <i>Présenter le module comme réponse au problème</i>	7	Diagrammes de séquence <i>Échanges entre composants pour chaque scénario</i>
3	L'argumentation <i>Persona · Interview · Customer Journey · Hypothèse · JTBD</i>	8	Diagrammes d'activité <i>(Optionnel) Processus avec conditions et boucles</i>
4	Priorisation MoSCoW <i>Must / Should / Could / Won't Have</i>	9	Diagramme de classes & Données <i>Classes UML · ERD 3NF · Schéma SQL · System of Record</i>
5	Walking Skeleton (MVP) <i>Minimum viable pour tester l'hypothèse</i>	10	Architecture — Composants <i>Blocs du module · Interfaces · Communication inter-modules</i>
		11	Choix technologiques <i>Langages · BDD (ACID/BASE) · APIs · Justification</i>

La logique interne — tout découle de tout

Un document cohérent : chaque section alimente la suivante



Ce qui est évalué, c'est la cohérence entre les sections — pas la longueur du document.

Chaque section doit découler logiquement de la précédente. Si l'hypothèse change, les User Stories changent. Si les User Stories changent, le modèle de données change.

§1-2	§3	§4-5	§6-8	§9-10	§11
Problème & Solution	Argumentation (CJ · Hypothèse)	MoSCoW & MVP	Use Cases & Séquences (Activité au besoin)	Modèle & Archi	Choix techno

Ce que chaque équipe a produit

11 sections du cahier des charges — état des lieux complet

Section	Titre	Contenu	Semaines
§1	Le problème	Description du problème réel	M1
§2	La solution	Description du module proposé	M1
§3.1	Le persona	Persona principal avec verbatims	M1
§3.2	L'interview	Résumé et verbatims clés	M1
§3.3	Le Customer Journey	Parcours complet · pain points · opportunités	M1
§3.4	L'hypothèse	Hypothèse testable au format 4 parties	M1
§3.5	Le JTBD	Job to be Done du persona principal	S5–S6
§4	Priorisation MoSCoW	Must / Should / Could / Won't Have	S5–S6
§5	Walking Skeleton	MVP end-to-end	S5–S6
§6	Use Cases & User Stories	Diagramme UML + US + critères Gherkin	S6–S7
§7	Diagrammes de séquence	Échanges entre composants par scénario	S7
§9	Classes & ERD	Diagramme de classes + ERD 3NF + SQL + System of Record	S8
§10	Architecture	Diagramme de composants + pattern justifié + style API	S9
§11	Choix technologiques	Stack · Sécurité · Performance · Accessibilité · ADRs	S9

Calendrier & Prochaines étapes

Projet 1 · Module 3 · Soutenances

Dates clés

Date	Événement
Lundi 1er juin	Aujourd'hui — récapitulatif complet Module 2
Jeudi 4 juin	Lancement officiel Projet 1 · rappel support S8 · questions
Vendredi 19 juin	Rendu définitif Projet 1 (35 %) — dépôt dans le dossier Teams
Semaine du 22 juin	Soutenances — 1h par groupe · format à confirmer

Module 3 — Travailler en équipe (début jeudi)

→ Manifeste Agile — valeurs et 12 principes	→ Git, branches, pull requests, revues de code
→ Scrum : sprints, backlog, daily, review, rétrospective	→ Definition of Done et Definition of Ready
→ Kanban : visualiser et limiter le travail en cours	→ Estimer, planifier et livrer de façon itérative

Ce que vous devez retenir

5 takeaways du cours

1

Le cycle de vie complet est votre carte

Vous êtes à l'étape 3 (conception terminée). Les étapes 4 à 6 arrivent dans les modules suivants.

2

Le Module 2 vous a donné tous les outils

De l'hypothèse au choix technologique, votre cahier des charges est le document de référence de tout ce qui suit.

3

La cohérence est ce qui est évalué

Chaque section doit découler de la précédente. Un cahier des charges incohérent, même bien rédigé, ne vaut pas grand-chose.

4

Le Projet 1 est votre spécification pour le Projet 2

Plus il est précis, plus l'agent IA produira du code juste au premier essai.

5

Jeudi, on lance officiellement

Préparez vos questions sur le rapport et relisez le support fourni en S8.

Jeudi 4 juin

Lancement officiel du Projet 1

- Relire le support S8 avant jeudi
 - Préparer vos questions sur le rapport
 - Compléter les sections §10 et §11 manquantes
-

GL-EN3-2026 · Jean Fritz SAINT-PAUL · FDS-UEH · Semaine 10